

Batch ingestion and the Glue ETL engine

The single most tested service on this exam, and the configuration details that decide the questions.

AWS Glue is not one service. It is **six things** sharing a name.

The six things called Glue

- ◆ **Glue ETL jobs.** Managed Apache Spark. The core of this module.
- ◆ **Glue Data Catalog.** Metadata store. Tables, columns, partitions.
- ◆ **Glue crawlers.** Scan data, infer schema, populate the catalog.
- ◆ **Glue Studio.** Visual authoring, including no-code data preparation.
- ◆ **Glue DataBrew.** Point-and-click cleaning for analysts, 250 plus transforms.
- ◆ **Glue Data Quality.** Rules that check data as it flows through.

ON THE EXAM

They are billed separately. A Data Catalog question and a Glue ETL question have nothing in common.

Where batch data comes from

01

Databases

On-premises or in AWS. DMS, Glue JDBC connections.

02

Files

Landing in S3, or arriving via DataSync or Transfer Family.

03

SaaS apps

Salesforce, ServiceNow, Zendesk. Amazon AppFlow.

04

Third parties

AWS Data Exchange, or a partner dropping into S3.

05

Physical

AWS Snowball, when the network cannot carry it.

ON THE EXAM

Almost all of these land in Amazon S3 first. That is the default correct landing zone.

Source to tool, at a glance

SOURCE	REACH FOR
Relational database, ongoing sync	AWS DMS with change data capture
Relational database, one-off read	Glue job with a JDBC connection
Files on NFS or SMB shares	AWS DataSync
Partner uploads over SFTP	AWS Transfer Family
SaaS application data	Amazon AppFlow
Petabytes, poor connectivity	AWS Snowball
Another company's dataset	AWS Data Exchange

ON THE EXAM

DMS for continuous replication. A Glue JDBC job for a scheduled extract. The difference is ongoing versus one-off.

Full load or incremental?

FULL LOAD

Reprocess everything

- Simple, and always correct
- Idempotent by construction
- Cost grows with total data
- Fine while the dataset is small

INCREMENTAL

Only what changed

- Needs a watermark or bookmark
- Cost grows with change volume
- Backfills need care
- The default at any real scale

THE TIEBREAKER

Glue job bookmarks are how AWS does incremental. They are the answer to "stop reprocessing old data".

A Glue job is an **Apache Spark application** that AWS starts, scales and shuts down for you.

Glue versions, and what they run

GLUE VERSION	ENGINE
Glue 5.0	Apache Spark 3.5.2, Python 3.11. Current
Glue 4.0	Apache Spark 3.3, Python 3.10
Glue 3.0	Apache Spark 3.1. First version with Auto Scaling and Flex
Glue 2.0	Spark 2.4. Introduced 1-minute minimum billing
Python shell	No Spark at all. 0.0625 or 1 DPU

ON THE EXAM

Glue 5.0 is roughly 32% faster than 4.0 on the same workload, at lower cost.

Worker types

WORKER	WHAT YOU GET
G.1X	1 DPU. 4 vCPU, 16 GB. The sensible default
G.2X	2 DPU. 8 vCPU, 32 GB. When G.1X runs out of memory
G.4X / G.8X	4 or 8 DPU. Demanding joins and aggregations
G.12X / G.16X	Largest general compute. Glue 4.0 and later
R.1X to R.8X	Double the memory per vCPU. Caching, shuffles, skew
G.025X	0.25 DPU. Streaming jobs only, low volume

ON THE EXAM

One DPU is 4 vCPU and 16 GB. R workers double the memory ratio, not the CPU.

How the bill is calculated

- ◆ **Billed per second** on DPU-hours consumed, with a one minute minimum.
- ◆ **Crawlers and dev endpoints** have a ten minute minimum. Different rule.
- ◆ **A Spark job defaults to 10 DPUs.** The minimum is 2. Most jobs need 3 to 5.
- ◆ **Flex** runs on spare capacity for up to 34 percent less.
- ◆ **Auto Scaling** bills only the workers actually used, not the maximum.

ON THE EXAM

The default 10 DPUs on a job that needs 3 is the most common source of a surprising Glue bill.

DynamicFrame or DataFrame?

GLUE DYNAMICFRAME

Schema is messy

- Handles inconsistent or unknown schemas
- resolveChoice for conflicting types
- relationalize flattens nested data
- Glue-specific, not portable Spark

SPARK DATAFRAME

Schema is known

- Standard Spark. Portable anywhere
- Faster, better optimised
- Needs a consistent schema
- Convert with toDF and fromDF

THE TIEBREAKER

Semi-structured, evolving or dirty data favours DynamicFrame. Clean known data favours DataFrame.

DynamicFrame transforms worth knowing by name

- ◆ **ApplyMapping.** Rename, cast and drop columns in one step.
- ◆ **ResolveChoice.** Decide what happens when one field has two types.
- ◆ **Relationalize.** Flatten nested JSON into related tables.
- ◆ **DropNullFields.** Remove columns that are entirely null.
- ◆ **FindMatches.** Machine learning transform for fuzzy record deduplication.

ON THE EXAM

Relationalize is the answer to "load nested JSON into a relational warehouse".

Job bookmarks: how incremental actually works

- ◆ **What they do.** Track what has already been processed between runs.
- ◆ **For S3,** they track object keys and timestamps.
- ◆ **For JDBC,** they need an increasing primary key or timestamp column.
- ◆ **They must be enabled.** Disabled is the default on a new job.
- ◆ **Rewind by resetting** the bookmark, or pause without losing state.

ON THE EXAM

Bookmarks need `job.init` and `job.commit` in the script. Without those calls they silently do nothing.

Job settings that decide questions

SETTING	WHY IT MATTERS
Number of workers	Defaults to 10 DPU. Most jobs need 3 to 5
Max retries	Retries a failed run. Only safe if the job is idempotent
Job timeout	Defaults to 48 hours. A runaway job bills the whole time
Max concurrent runs	Defaults to 1. A second trigger is rejected, not queued
Temporary directory	An S3 path Glue writes scratch data to. Needs a lifecycle rule
Job parameters	Key-value arguments. Where bookmarks and Auto Scaling are switched on

ON THE EXAM

Max concurrent runs of 1 is why an event-triggered job silently drops overlapping files.

Standard or Flex execution?

STANDARD

Starts immediately

- Dedicated capacity, predictable start
- Use when there is an SLA
- Full price
- Any Glue version

FLEX

Starts eventually

- Runs on spare AWS capacity
- Start and finish times vary
- Up to 34 percent cheaper
- Glue 3.0+, Spark ETL jobs only

THE TIEBREAKER

Flex is the answer whenever a question says the job is not time-sensitive and cost matters.

Auto Scaling

- ◆ **You set a maximum**, and Glue uses only what the job needs.
- ◆ **You are billed** for workers actually running, not the ceiling.
- ◆ **Requires Glue 3.0** or later. Off by default.
- ◆ **Best for variable workloads** where data volume differs run to run.
- ◆ **Not a fix for a badly written job.** Skew stays skewed at any scale.

ON THE EXAM

Auto Scaling and Flex stack. A non-urgent job with variable volume should use both.

A company runs a nightly AWS Glue job that transforms the previous day's order files in Amazon S3 and writes Parquet output. The job was created eighteen months ago and has never been reconfigured. It uses the default settings.

The job now takes four hours and the monthly Glue bill has grown steadily. Finance has asked for a reduction. The job must finish before the business day begins at eight in the morning, and it currently starts at midnight. Data volume grows by about two percent a month.

Which combination will reduce cost with the least risk to the deadline?

- | | |
|---|---|
| <p>A Switch the job to the Flex execution class and enable job bookmarks</p> | <p>B Increase the worker type from G.1X to G.8X so the job finishes sooner</p> |
| <p>C Reduce the DPU allocation from the default of 10 to 2 and accept a longer runtime</p> | <p>D Migrate the job to Amazon EMR on EC2 with Spot Instances</p> |

◆ PAUSE AND DECIDE

Which combination will reduce cost with the least risk to the deadline?

- A

 Flex plus job bookmarks
Flex cuts the rate by up to 34 percent. Bookmarks stop reprocessing old files. Eight hours of slack absorbs Flex's variable start.
- B

 Move to G.8X workers
Finishing sooner on eight times the DPU's costs the same or more. Speed was not the problem.
- C

 Drop to 2 DPUs
Tempting and reckless. Two DPUs is the floor; a four-hour job could exceed the eight-hour window.
- D

 EMR on EC2 with Spot
Cheaper per unit, and swaps a serverless job for a cluster to operate. Nothing suggested they want that.

THE SIGNAL

"Never been reconfigured" and "default settings" are the planted details. Defaults are the problem.

Read from the catalog, or from a path?

FROM THE DATA CATALOG

Schema already registered

- Partitions are known in advance
- Pushdown predicates can prune them
- Lake Formation permissions apply
- Needs a crawler or registered table

DIRECT FROM S3

Schema inferred each run

- No catalog entry required
- Inferred on every read
- No partition metadata to prune with
- Fine for one-off or transient data

THE TIEBREAKER

Catalog reads are what make partition pruning and fine-grained permissions possible at all.

Writing output well

- ◆ **partitionKeys.** Writes Hive-style folders later queries can prune.
- ◆ **enableUpdateCatalog.** The job registers new partitions itself, no crawler needed.
- ◆ **Parquet with Snappy** is the default correct output for analytics.
- ◆ **Overwrite carefully.** A failed overwrite can leave a partition half-written.
- ◆ **Match partition cardinality** to how data is queried, not how it arrives.

ON THE EXAM

A job that updates the catalog as it writes removes an entire scheduled crawler from the design.

Glue and open table formats

- ◆ **Apache Iceberg, Apache Hudi and Delta Lake** are natively supported.
- ◆ **Iceberg** is the format the exam guide now names explicitly.
- ◆ **Row-level updates and deletes** become possible on data in S3.
- ◆ **Time travel** lets a job read the table as it stood at a past moment.
- ◆ Enabled with a job parameter naming the data lake framework.

ON THE EXAM

Version 1.1 of the exam guide added open table formats. Older question banks do not test this.

When a Glue job fails, it is more often a **network or permissions** problem than a code problem.

How Glue reaches a private data source

- ◆ **A Glue connection** holds the VPC, subnet and security group to use.
- ◆ **Glue creates elastic network interfaces** in that subnet to reach the source.
- ◆ **The security group must allow itself.** A self-referencing inbound rule is required.
- ◆ **The subnet needs a route to S3,** or the job cannot read its own script.
- ◆ **Credentials** belong in AWS Secrets Manager, not in the job script.

ON THE EXAM

A job in a VPC with no S3 route fails before running a line of your code.

Glue connection types

CONNECTION TYPE	USED FOR
JDBC	Relational databases, in AWS or on-premises
Network	Placing the job in a VPC where no driver is needed
Kafka	Amazon MSK or self-managed Apache Kafka
MongoDB / DocumentDB	Document stores
Marketplace and custom	Third-party connectors, including SaaS sources

ON THE EXAM

A Network connection exists purely to place the job inside your VPC. It carries no driver.

Gateway or interface endpoint?

GATEWAY ENDPOINT

S3 and DynamoDB only

- Added as a route table entry
- No hourly charge
- Cannot be reached from on-premises
- The usual answer for S3 from Glue

INTERFACE ENDPOINT

Almost everything else

- An ENI with a private IP, via PrivateLink
- Charged hourly plus data processed
- Works from on-premises over VPN or Direct Connect
- Needed for Glue, KMS, Secrets Manager

THE TIEBREAKER

A gateway endpoint that is not in the subnet's route table does nothing at all.

A data engineer has configured an AWS Glue job to read from an Amazon S3 bucket. A Glue connection has been created with the correct VPC, private subnet and security group, and the IAM role attached to the job grants full access to the bucket.

The job fails on every run with an error referring to the Amazon S3 VPC gateway endpoint. The bucket policy is permissive and the IAM role has been verified as correct.

What is the most likely cause?

- ☐ A The Glue security group does not allow inbound traffic from the S3 gateway endpoint
- ☐ B The bucket policy does not explicitly name the Glue service principal
- ☐ C The Glue connection is missing a fully qualified domain name
- ☐ D The subnet's route table has no route to the S3 gateway endpoint

◆ PAUSE AND DECIDE

What is the most likely cause?

- A

 Security group inbound from the endpoint
A gateway endpoint is a route, not a source of traffic. There is no address to allow inbound from.
- B

 Bucket policy missing the service principal
The question already told you IAM and the bucket policy are correct. It is ruling this out for you.
- C

 Missing fully qualified domain name
Invented. FQDNs matter for some JDBC connections, not for S3 access through an endpoint.
- D

 No route in the subnet's route table
A gateway endpoint works only when the subnet's route table sends S3 traffic to it. Nothing else applies it.

THE SIGNAL

Options that restate a fact the question already gave you as verified are almost never the answer.

Finding sensitive data

- ◆ **Glue Sensitive Data Detection** identifies PII as data flows through a job.
 - ◆ Can redact, mask, hash or partition out the values it finds.
- ◆ **Amazon Macie** scans data at rest in Amazon S3, on a schedule.
 - ◆ Macie does not sit inside a pipeline. Glue detection does.
- ◆ Both appear in the same question. The difference is when they run.

ON THE EXAM

In-flight detection is Glue. At-rest discovery across buckets is Macie.

Seeing inside a running job

- ◆ **Continuous logging** streams driver and executor logs to CloudWatch during the run.
- ◆ **The Spark UI** shows stages and tasks, and where the time actually went.
- ◆ **Job run insights** surfaces the failing line and a suggested root cause.
- ◆ **CloudWatch metrics** expose executor counts, memory and shuffle bytes.
- ◆ Without continuous logging, logs appear only once the job ends.

ON THE EXAM

A four-hour job that fails at hour three, with no logs until it finishes, is the case for continuous logging.

Authoring surfaces, and who they are for

- ◆ **Glue Studio visual ETL.** Drag-and-drop boxes, generates the Spark code.
- ◆ **Glue Studio data preparation.** Spreadsheet-style, no code, runs on Glue Spark.
- ◆ **Glue notebooks and interactive sessions.** Jupyter against live Glue compute.
- ◆ **Script editor.** Write PySpark or Scala directly.
- ◆ **Glue DataBrew.** Separate service, for analysts, recipe-based.

ON THE EXAM

DataBrew recipes can be imported into Glue Studio data preparation and run at Glue job prices.

Two Glue job types that are not Spark

- ◆ **Python shell jobs.** Plain Python, 1 DPU or one-sixteenth of a DPU.
- ◆ Right for small scripts, API calls, single SQL statements, file moves.
- ◆ **Glue for Ray.** Closed to new customers from the thirtieth of April, twenty twenty six.
- ◆ AWS recommends Amazon EKS as the alternative for Ray workloads.
- ◆ Existing Ray customers continue as normal. It may still appear in questions.

ON THE EXAM

Availability warning: Glue for Ray is closed to new sign-ups. Learn it, do not plan on it.

Starting Glue jobs

- ◆ **Schedule.** A cron expression on the job, or a Glue trigger.
- ◆ **On-demand.** StartJobRun from an SDK, the CLI, or another service.
- ◆ **Conditional trigger.** Fires when other jobs or crawlers succeed or fail.
- ◆ **Glue workflow.** Chains crawlers, jobs and triggers into one unit.
- ◆ **EventBridge.** Fires from an S3 upload, or any other AWS event.

ON THE EXAM

Glue workflows handle Glue-only pipelines. Step Functions handles pipelines that reach beyond Glue.

Making reruns safe

- ◆ **Overwrite a partition** rather than appending, so a rerun replaces its own output.
- ◆ **Bookmarks** make a rerun skip what was already consumed.
- ◆ **Iceberg merge** upserts by key, so duplicates collapse rather than accumulate.
- ◆ **Blind append plus retries** is how a pipeline silently duplicates data.
- ◆ Enable retries only once one of those three is in place.

ON THE EXAM

Retries and appends together are the classic cause of row counts that grow on every failure.

Glue jobs get slow for **four reasons**, and only one of them is fixed by adding workers.

Symptom to cause to fix

SYMPTOM	CAUSE AND FIX
All executors busy, job just slow	Under-provisioned. Add workers or enable Auto Scaling
One task runs long, others idle	Data skew. Repartition, salt the key, or use an R worker
Out of memory on the driver	Collecting too much to the driver, or a huge query plan
Out of memory on executors	Move up a G size, or sideways to an R type
Slow before any processing starts	Small files. Millions of objects to list and open
Reads far more data than needed	No pushdown predicate. Add partition filters

ON THE EXAM

Skew is the one where adding capacity provably does not help. Extra executors sit idle.

Pushdown predicates and partition filters

- ◆ **Without one**, Glue reads everything and filters in memory afterwards.
- ◆ **With one**, Glue never opens the partitions it does not need.
- ◆ **push_down_predicate** filters on partition columns before reading.
- ◆ **catalogPartitionPredicate** pushes the filter into the Data Catalog itself.
- ◆ Filtering after the read is the single most wasteful Glue mistake.

ON THE EXAM

This only works if the data is partitioned on the column you filter by.

The small file problem, and its opposite

- ◆ **Millions of tiny objects.** Listing and opening costs more than reading.
- ◆ **Fix on write:** coalesce or repartition before writing output.
- ◆ **Fix on read:** enable grouping so Glue batches small files per task.
- ◆ **Fix structurally:** a compaction job, or Iceberg's managed compaction.
- ◆ **The opposite failure:** a few enormous unsplittable files, so no parallelism.

ON THE EXAM

Aim for files in the hundreds of megabytes. Both extremes hurt, for opposite reasons.

A Glue job reads a partitioned Parquet dataset in Amazon S3 containing five years of transaction history, partitioned by year and month. The job produces a report covering the most recent month only.

The job runs on twenty G.2X workers and takes ninety minutes. Monitoring shows all executors are busy throughout, and the job reads roughly four terabytes on every run. The team has been asked to reduce both runtime and cost without changing the report.

What should the data engineer do first?

- ☐ A Increase the number of workers from twenty to sixty to shorten the runtime
- ☐ B Add a pushdown predicate so the job reads only the current month's partitions
- ☐ C Switch the workers from G.2X to R.2X to provide more memory per worker
- ☐ D Enable Auto Scaling with a maximum of twenty workers

◆ PAUSE AND DECIDE

What should the data engineer do first?

- A

 Triple the workers
All executors are already busy, so this genuinely would be faster. It would also roughly triple the cost. The question asked for both.
- B

 Add a pushdown predicate
Reads one month instead of sixty. Cuts runtime and cost together, because Glue never opens the other partitions.
- C

 Move to R.2X workers
More memory solves memory pressure. Nothing here reports an out-of-memory failure.
- D

 Auto Scaling capped at twenty
Scales down idle workers. Every executor is busy, so there is nothing idle to reclaim.

THE SIGNAL

Compare with scenario 1. There the fix was configuration. Here it is the code, because the job reads data it never uses.

An AWS Glue job loads a nightly extract into a curated Amazon S3 dataset. Max retries is set to three because the source database occasionally times out. The job appends its output to the current day's partition.

Analysts report that on some days the revenue totals are roughly double what finance expects. The affected days correlate with nights when the source database was slow. The job reports success on every one of those nights.

What is happening, and what is the correct fix?

- | | |
|--|--|
| <p>A The source database is sending duplicate rows. Add a DISTINCT to the extract query</p> | <p>B Retries re-run a job that had already written output. Write with overwrite on the partition instead of appending</p> |
| <p>C Max retries should be set to zero so that failures are surfaced to the operations team</p> | <p>D The job needs more DPUs so that it completes before the database times out</p> |

◆ PAUSE AND DECIDE

What is happening, and what is the correct fix?

- A

 Add DISTINCT to the extract
Treats a symptom in the wrong place. The source is fine; the duplication is created downstream by the rerun.
- B

 Overwrite the partition instead of appending
Makes the rerun replace its own earlier output. The job becomes idempotent and retries become safe.
- C

 Set max retries to zero
Removes the duplication and the resilience with it. Every transient timeout now becomes a failed night.
- D

 Add more DPUs
The timeout is on the source database. More Glue workers do not make someone else's database faster.

THE SIGNAL

"Reports success" plus "doubled totals" is the signature of a retry over a non-idempotent write.

Glue or Lambda for a batch job?

AWS LAMBDA

Small and quick

- 15 minute maximum execution
- Up to 10 GB memory, 10 GB ephemeral storage
- No cluster, starts in milliseconds
- Per-file processing, light transforms

AWS GLUE

Large or distributed

- No practical runtime limit
- Scales across many workers
- Spark, joins across large datasets
- Minute-plus startup

THE TIEBREAKER

The 15 minute Lambda ceiling is the usual decider. If the job might exceed it, Lambda is out.

Glue or EMR?

AWS GLUE

Serverless Spark

- No cluster to manage or tune
- Spark and Python only
- Glue-specific extras like DynamicFrames
- Best for scheduled ETL

AMAZON EMR

The full ecosystem

- Hive, Presto, HBase, Flink, Trino too
- Deep cluster and Spark tuning available
- Spot Instances can cut cost sharply
- EMR Serverless removes the cluster

THE TIEBREAKER

Existing Hadoop or Hive workloads, or a need for engines other than Spark, point at EMR.

Sometimes the answer is no pipeline at all

BUILD A PIPELINE

Glue, DMS, orchestration

- Full control over transformation
- Works between any two systems
- Code, schedules and failures to own
- Right when data must change shape

ZERO-ETL INTEGRATION

AWS keeps them in sync

- Aurora, RDS and DynamoDB into Redshift
- Near real-time, operated by AWS
- No job, no schedule, no code
- Supported source and target pairs only

THE TIEBREAKER

Zero-ETL is the lowest-overhead answer when the pair is supported and no transformation is needed.

When Glue is the wrong answer

- ◆ **Sub-second latency.** Glue jobs start in minutes, not milliseconds.
- ◆ **A single small file, frequently.** Lambda is cheaper and faster.
- ◆ **Non-Spark engines needed.** Hive, Presto, HBase mean EMR.
- ◆ **Pure SQL over the warehouse.** Do it in Redshift, do not export and reimport.
- ◆ **Continuous database replication.** That is AWS DMS with CDC.

ON THE EXAM

The last one matters. Glue can read a database; it is not a replication tool.

A financial services company receives one CSV file from each of forty partner banks every night, between two and eight megabytes each. Each file needs the same light transformation: three columns renamed, one date reformatted, and the result written as Parquet to Amazon S3.

Files arrive at unpredictable times through the night. The current design runs a single Glue Spark job at six in the morning that processes whatever has arrived. The team wants results available as soon as each file lands, and wants to reduce cost.

Which design best meets both requirements?

- ☐ A Keep the Glue job but schedule it hourly instead of once at six
- ☐ B Trigger an AWS Lambda function from S3 Event Notifications to transform each file on arrival
- ☐ C Trigger a Glue Spark job from S3 Event Notifications for each file that arrives
- ☐ D Replace the design with an Amazon EMR cluster running continuously through the night

◆ PAUSE AND DECIDE

Which design best meets both requirements?

- A

 Hourly Glue schedule
Improves latency a little, and adds runs that find nothing. Cost goes up, not down.
- B

 Lambda per file on arrival
Megabytes, a rename and a date format, well inside 15 minutes. Event-driven, no idle cost, milliseconds to start.
- C

 Glue job per file on arrival
Correct trigger, wrong engine. Forty Spark job starts, each billed a one minute minimum, to process 8 MB.
- D

 EMR running all night
A cluster idle most of the night, for work a function does in seconds. Wrong on both requirements.

THE SIGNAL

Small file, light transform, event-driven. This is the shape where Glue loses to Lambda.

A healthcare analytics company runs a Glue job that reads patient records from Amazon S3, joins them with reference data, and writes a curated dataset used by three analytics teams. Compliance requires that one team can see all columns, and the other two must not see the patient identifier column at all.

The job currently reads directly from an S3 path using `create_dynamic_frame.from_options`. The security team has asked for column-level access control without maintaining three separate copies of the data.

What should the data engineer change?

- ☐ A Write three output datasets to different S3 prefixes and control access with bucket policies
- ☐ B Register the dataset in the Glue Data Catalog, read from the catalog, and grant column-level permissions in AWS Lake Formation
- ☐ C Encrypt the patient identifier column with AWS KMS and grant key access only to the permitted team
- ☐ D Enable Glue Sensitive Data Detection on the job to redact the patient identifier

What should the data engineer change?

- A

 Three copies with bucket policies
Explicitly ruled out by the question, which asked for no duplicate copies. It also triples storage and drift risk.
- B

 Catalog the table, grant columns in Lake Formation
Lake Formation grants apply to catalog tables, so the read path has to change too. One copy, three views.
- C

 Encrypt the column with KMS
Client-side column encryption is real, and it is a heavy custom design here. It also breaks joins and analytics on that column.
- D

 Sensitive Data Detection to redact
Redacts for everyone. The requirement is that one team must still see it, which redaction cannot express.

THE SIGNAL

Reading from a path bypasses Lake Formation entirely. Fine-grained control requires the catalog read path.

Reading a Glue question quickly

- ◆ **Which Glue?** ETL job, catalog, crawler, Studio or DataBrew.
- ◆ **How big, how long?** Under 15 minutes and small means Lambda is live.
- ◆ **Is there an SLA?** No deadline plus cost pressure means Flex.
- ◆ **Is it reprocessing?** Growing cost with static volume means bookmarks.
- ◆ **Is it reading too much?** Pushdown predicates before more workers.

ON THE EXAM

Next: streaming ingestion. Kinesis, Data Firehose, MSK and Managed Service for Apache Flink.